

Zurich ServiceNow AI Platform Capabilities

Last updated: January 16, 2026

Some examples and graphics depicted herein are provided for illustration only. No real association or connection to ServiceNow products or services is intended or should be inferred.

This PDF was created from content on docs.servicenow.com. The web site is updated frequently. For the most current ServiceNow product documentation, go to docs.servicenow.com.

Company Headquarters

2225 Lawson Lane
Santa Clara, CA 95054
United States
(408)501-8550

Integration Commons for CMDB

The Integration Commons for CMDB (sn_cmdb_int_util) store app contains the CMDB Integrations Dashboard and a set of Robust Transform Engine (RTE) transforms and script includes.

Request apps on the Store

Visit the [ServiceNow Store](#) website to view all the available apps and for information about submitting requests to the store. For cumulative release notes information for all released apps, see the [ServiceNow Store version history release notes](#).

Dependencies

CMDB and CSDM Data Foundations Dashboards v2.3.5 or later versions (see [Monitor health in CSDM and CMDB Data Foundations Dashboards](#))

Using the CMDB Integrations Dashboard

The Integration Commons for CMDB store app provides a dashboard with a central view of the status, processing results, and processing errors of all installed Service Graph Connectors and any custom integrations created in IntegrationHub ETL.

See [CMDB Integrations Dashboard](#) for more information.

Using RTE transforms

The Integration Commons for CMDB (com.snc.cmdb.integration_util) plugin provides the Integration Commons functionality. You can use the transforms and script includes to standardize the values stored in the CMDB by different data integrations or by changes. The attributes that are included in the Integration Commons for CMDB store app are attributes that the Identification and Reconciliation Engine (IRE) requires for identification or attributes that could be used to derive classes.

The transforms are common operation types and template scripts available as templated operations. A templated operation is a transform that controls the logic for the transform. The result is that there can be only a single output. When a transform returns multiple values, then those values are concatenated by a triple pipe (| |). You then must use the

split transform to retrieve the values that you're interested in. The inputs are either a single field or a list of fields. For all but one transform, the inputs are assumed to be a fixed list of fields as described for each of the following individual transform.

Note: The RTE transforms are included in the Integration Commons for CMDB store app and are available in the [IntegrationHub ETL store app](#). For more information on RTE transforms, see [RTE operation types included within the Integration Commons for CMDB app](#) and [RTE transforms template scripts included within the Integration Commons for CMDB app](#).

ServiceNow Service Graph Connectors that are available at the ServiceNow Store, have dependencies on the transforms and script includes in the Integration Commons for CMDB store app. Therefore, when you install such CMDB integration, the Integration Commons for CMDB store app is automatically installed too.

You can also configure the Application Dependency Mapping (ADM) adapter to populate running processes, TCP connections, and applications into CMDB. For more information, see [Configuring the ADM adapter for Service Graph Connectors](#).

Important: After upgrades and deployments of new applications or integrations, run quick start tests to verify that Integration Commons for CMDB works as expected. See [Quick start tests for Integration Commons for CMDB](#) for more information.

Related concepts

- [Configuring the ADM adapter for Service Graph Connectors](#)
- [Accessing the connection details of Service Graph Connectors](#)
- [Managing CMDB data deletion](#)
- [Partition size computation for parallel loading in Integration Commons for CMDB](#)

Related reference

- [Quick start tests for Integration Commons for CMDB](#)

CMDB Integrations Dashboard

You can use the CMDB Integrations Dashboard that provides a central view of status, processing results, and processing errors of all installed Service Graph Connectors and any custom integrations created in IntegrationHub ETL run.

Note: Starting with the Xanadu release, the CMDB Integrations Dashboard is available in the Next Experience user interface. For more information on the Next Experience user interface, see [Next Experience UI](#).

On the CMDB Integrations Dashboard, you can see metrics for all Service Graph Connector runs, or filter the view to a specific connector, a specific time duration, or a specific connector run.

Learn more about the CMDB Integrations Dashboard from the following video.

Access CMDB Integrations Dashboard

Access the CMDB Integrations Dashboard provided with the Integration Commons for CMDB store app for maintaining data consistency and accuracy across multiple data sources.

Before you begin

Role required: None

Procedure

1. Navigate to **All > Self-Service > Dashboards**.

Tip: Alternatively, you can navigate to **All > Platform Analytics > Library > Dashboards**.

2. On the Dashboards view, select **CMDB Integrations Dashboard** and perform any of the following tasks:

- Select the **CMDB Execution Status** tab to see metrics such as the total number of integrations and processed rows, integration runs actively running, daily statistics, and details about the classes that were updated.
- Select the **CMDB Integration Errors** tab to see metrics such as number of import and integration errors, and number of erroneous imported records.
- Point to the score on the various tiles to drill down to the list views for the associated records. Point to the charts to show more details for the chart.
- Narrow down the scope of the integration runs included in the metrics on the dashboard by configuring filters on the right-hand side of the dashboard. Set any of the following filters and then select **Apply**. The filter settings apply to any metric with a filter icon in its upper left corner.

Filters

Filter	Description
Import Date	Select All or a time period, such as Last 7 days , from which to include integration runs in metrics.
CMDB Applications	Select All , or a CMDB application, such as SCCM , or a custom integration, from which to include integration runs in metrics.
CMDB Import	Select All , or a specific integration run to include in metrics.

RTE operation types included within the Integration Commons for CMDB app

The Robust Transform Engine (RTE) operation types are common operation methods for use in ETL without having to write your own complex data transformations.

The following operation types are available in the Integration Commons for CMDB app.

Concatenation

Combines the values from input fields into a single string, joining them on the optional joining_string field.

Details	
Table	RTE Entity Concatenation Operation [sys_rte_eb_concat_operation]
Input field	source_sys_rte_eb_fields Input is a set of fields and constants.
Output field	target_sys_rte_eb_field Output is the combined value of the provided fields as a single string.
Additional fields	joining_string (optional)

Example

Input	joining_string	Result
"input_1", "input_2", "input_3"	" , "	"input_1, input_2, input_3"

Convert to Boolean

Converts the incoming value to a Boolean.

Details	
Table	RTE Entity Convert to Boolean Operation [sys_rte_eb_to_boolean_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_field Output is <code>true</code> when input is <code>true</code> or <code>1</code> , otherwise output is <code>false</code> . (case insensitive)

Example

Input	Result
input_1	false
true	true
1	true
0	false
11	false

Convert to Date

Attempts to convert the incoming value to a GlideDateTime value by applying the `date_format` to the incoming value.

Details	
Table	RTE Entity Convert to Date Operation [sys_rte_eb_to_date_operation]
Input field	source_sys_rte_eb_field Input is a data timestamp value with date format.
Output field	target_sys_rte_eb_field Output is the date timestamp in the specified date format. Attempts to directly convert using GlideDateTime if the date_format is incorrect. Returns an empty value if unable to parse at all.

Example

Input	Result
"2018/09/20 11:21:00 AM EST" with date_format "yyyy/MM/dd hh:mm:ss a z"	"2018-09-20 16:21:00"
"2018/09/20 01:21:00 PM EST" with date_format "yyyy/MM/dd hh:mm:ss a z"	"2018-09-20 18:21:00"
"09/20/18" with date_format "yyyy/MM/dd hh:mm:ss a z"	""0018-09-20 00:00:0"

Convert to Numeric

Converts the incoming value to a number.

Details	
Table	RTE Entity Convert to Numeric Operation [sys_rte_eb_to_numeric_operation]
Input field	source_sys_rte_eb_field Input is a value.
Output field	target_sys_rte_eb_field Output is a numeric value. If the input value is non-numeric the output is empty.

Example

Input	Result
input_1	null
1.23	1.23
1.00	1
two	null

Copy

Copies the value of the source field to all the target fields.

Details	
Table	RTE Entity Copy Operation [sys_rte_eb_copy_operation]
Input field	source_sys_rte_eb_field Input is a value.

Details	
Output field	target_sys_rte_eb_fields Output is the copied source field value.
Additional field	overwrite_existing_value (optional, Boolean): If <code>true</code>, then the values of the target fields are replaced. Otherwise, any non-empty value isn't overwritten.

Extract First Numeric

Sets the target field as the first numeric value found in the source field.

Details	
Table	RTE Extract Numeric Operation [sys_rte_eb_extract_numeric_operation]
Input field	source_sys_rte_eb_field Input is a value.
Output field	target_sys_rte_eb_field Output is the numeric value found in the input.
Additional fields	- decimal_places (optional, number) : Forces the output to have a specified number of decimal places. - remainder_target_field (optional, reference to field): Set to the trimmed remainder of the source field, after removing the first numeric value.

Example

Input	Result
100 mb	100
100.123 mb	100.123
100.123 mb with decimal_places=2	100.12
100 mb with decimal_places=2	100.00
100 mb with remainder_target_field	mb

Glide Lookup Operation

Performs a lookup in the database on the target table specified in the target_table field.

Details	
Table	RTE Glide Lookup Operation [sys_rte_eb_glide_lookup_operation]
Input field	source_sys_rte_eb_fields The database table for lookup.
Output field	target_sys_rte_eb_fields The resulting data based on the lookup operation.
Additional fields	- target_table - glide_matching_fields (string): Comma-separated list of column names in the target table. For each input field in source_sys_rte_eb_fields, there must be an equal number of values in glide_matching_fields.

Details	
	glide_target_fields (string): Comma-separated list of column names in the target table. For each target field in target_sys_rte_eb_fields, there must be an equal number of values in glide_target_fields.

Example

Input	Result
- Input Field 1: 100 South Charles Street, Baltimore - Input Field 2: MD - Target Table: Location (cmn_location) - Glide Matching Fields: street,state - Glide Target Fields: sys_id	Output Field 1: 25ab9c4d0a0a0bb300f7dabdc0ca7c1c

Multiple Input Script

Runs a script with multiple inputs setting the target_sys_rte_eb_field field as the output for that script.

Details	
Table	RTE Entity Multiple Input Script Operation [sys_rte_eb_multi_in_script_operation]
Input field	source_sys_rte_eb_fields Input is a script.

Details	
Output field	target_sys_rte_eb_field Output is the result of the input script.
Additional Fields	- script (script) - use_unique_input_sets(Boolean): When <code>true</code>, only unique input values are included in the data batch for IRE processing. Otherwise, all input object's field values are included.

Example for using `use_unique_input_sets`, with a script function that takes `record_type` and `operating_system` as input and returns `record_with_os`:

Examples

Record	record_type	operating_system
1	computer	Windows XP
2	computer	Linux
3	computer	Windows XP

If `use_unique_inputs_sets` is set to `true`, then the script processes only two values (`computer + Windows XP` and `computer + Linux`). If `use_unique_inputs_sets` is set to `false`, then each of the three values is individually processed (`computer + Windows XP`, `computer + Linux`, and `computer + Windows XP`).

Sample script:

```
(function(batch, output) {
    for (var i = 0; i < batch.length; i++) {
        // batch[i] is the unique set of
        inputs/individual record
    }
})
```

```

// batch[i].<field> gives access to the field value
var in0 = gs.nil(batch[i].record_type) ? '' : batch[i].record_type;
var in1 = gs.nil(batch[i].operating_system) ? '' : batch[i].operating_system;
// output[i] is the output for the specific combination of inputs/individual record
output[i] = in0 + "_" + in1;
    }
  }
})(batch, output);

```

Multiple Input/Output Script

Runs a script with multiple inputs setting the target fields specified in the `target_sys_rte_eb_fields` field as the multiple outputs for that script.

Details	
Table	RTE Entity Multiple Input/Output Script Operation [sys_rte_eb_multiple_input_output_script_operation]
Input field	source_sys_rte_eb_fields Input is a script.
Output field	target_sys_rte_eb_fields Output is the result of the input script.
Additional Fields	script (script)

Sample script:

```

(function(batch, output) {
    for (var i = 0; i < batch.length; i++) {
        var userId = (batch[i].user_id);
        var userIdParts = userId.split(".");
    }
});

```

```

                                output[i].first_name = userIdParts
[0];
                                output[i].last_name = userIdParts
[1];
                                }
                                }
                                })(batch, output);

```

Regex Replace

Replaces each substring of the input string that matches the regular expression pattern specified in the match_regex field with the string specified in the replacement_regex field.

Details	
Table	RTE Entity Regular Expression Replace Operation [sys_rte_eb_regex_replace_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_field Output is the replaced string.
Additional fields	- match_regex (string, regular expression) - replacement_regex (string)

Example

Input	Result
"String&With(Special)\$Characters" with match_regex="[^\0-9a-zA-Z]+" and replacement_regex=" "	"String With Special Characters"

Replace

Replaces each substring of the input string that matches the string specified in the match_string field with the string specified in the replacement_string field.

Details	
Table	RTE Entity Replace Operation [sys_rte_eb_replace_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_field Output is the replaced string.
Additional fields	- match_string (string) - replacement_string (string)

Example

Input	Result
"Original String" with match_string = "Original" and replacement_string= "Replacement"	"Replacement String"

Round Numeric

Rounds off the input numeric value to the nearest whole number. Non-numbers are truncated.

Details	
Table	RTE Entity Round Numeric Operation [sys_rte_eb_round_numeric_operation]
Input field	source_sys_rte_eb_field Input is a numeric value.
Output field	target_sys_rte_eb_field Output is a whole number.
Additional fields	- match_string (string) - replacement_string (string)

Example

Input	Result
"1.5"	"2"
"1.4"	"1"
"I'm a string"	""

Script Operation

Runs a script with an input and sets the target_sys_rte_eb_field field as the output for that script.

Note: The Script operation type has been largely superseded by the Multi Input Script Operation and is included for backwards compatibility with the existing configurations.

Details	
Table	RTE Entity Script Operation [sys_rte_eb_script_operation]
Input field	source_sys_rte_eb_field Input is a script.
Output field	target_sys_rte_eb_field Output is the result of the input script.
Additional fields	- script(script) - use_unique_input_sets (Boolean): When <code>true</code>, only unique input values are included in the data batch for IRE processing. Otherwise, all input object's field values are included. For an example and for more details, see the Multiple Input Script transform.

The source field is included in the 'batch' variable as the JavaScript field 'input'.

```
(function(batch, output) {
    for (var i = 0; i < batch.length; i++) {
        // batch[i] is the unique set of
        // inputs/individual record
        // batch[i].input gives access to
        // the field value
        var in0 = gs.nil(batch[i].input)
        ? '' : batch[i].input;
        // output[i] is the output for the
        // specific combination of inputs/individual record
        output[i] = in0 + " modified by script";
    }
})(batch, output);
```

Example:

```

/* Example Script
(function(batch, output) {
    for (var i = 0; i < batch.length; i++) {
        //step1: access the input variables
        var a = batch[i].input; //Value of the source fie
ld.

        //step2: Your script/code goes here.
        var b = a + 1;
        //step3: set the output for each elements
        output[i] = b;
    }
})(batch, output);
*/

```

Set

Sets the target_sys_rte_eb_field field value to the string provided in the set_value field.

Details	
Table	RTE Entity Set Operation [sys_rte_eb_set_operation]
Output field	target_sys_rte_eb_field Output is the value associated with the set_value field.
Additional fields	• set_value (string) • overwrite_existing_value (optional, Boolean) : When true, the current value of the target field is overwritten. Otherwise, a non-empty value isn't replaced.

Set Min/Max

Sets the target field to either the maximum or minimum of the values from all input fields.

Details	
Table	RTE Entity Min/Max Operation [sys_rte_eb_min_max_operation]
Input field	source_sys_rte_eb_fields Input is a set of values.
Output field	target_sys_rte_eb_field Output is the maximum or minimum value based on the min_max value.
Additional fields	- data_type (choice list with values as STRING, NUMERIC, and DATE) - min_max (choice list with values as MIN and MAX)

Examples

Input	Output
"2", "-1", "0"	2
"a", "b"	c
"2", "-1", "0"	-1
"a", "b"	a

Split

Splits the string included in the source_sys_rte_eb_field input value at the separator specified in the splitting_string field and assigns the resulting array of strings to the target_sys_rte_eb_field field, in order.

Details	
Table	RTE Entity Split Operation [sys_rte_eb_split_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_fields Output is list of substrings.
Additional fields	splitting_string (string)

Example

Input	Result
"value1 value2 value3", splitting_string:" " with target_sys_rte_eb_fields {target1,target2,target3}	target1 : value1, target2 : value2, target3 : value3
"value1 value2 value3", splitting_string:" " with target_sys_rte_eb_fields {target1}	target1 : value1
"value1", splitting_string:" " with target_sys_rte_eb_fields {target1,target2,target3}	target1 : value1, target2 : <null>, target3 : <null>

Trim

Removes any whitespaces at the beginning and at the end of the string included in the source_sys_rte_eb_field input value and assigns the result to the target_sys_rte_eb_field field. This transform is equivalent to the Java String trim() Method.

Details	
Table	RTE Entity Trim Operation [sys_rte_eb_trim_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_field Output is the input string value but without any leading and trailing spaces.

Example

Input	Result
" value 1 "	"value 1"

Uppercase

Changes all characters of the source_sys_rte_eb_field input value to upper case and assigns the result to the target_sys_rte_eb_field field.

Details	
Table	RTE Entity Upper Case Operation [sys_rte_eb_upper_case_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_field Output is the upper case string value.

Example

Input	Result
"value1"	"VALUE1"

Uppercase Trim

Combines both the Uppercase and the Trim transforms.

Details	
Table	RTE Entity Upper Case Trim Operation [sys_rte_eb_upper_case_trim_operation]
Input field	source_sys_rte_eb_field Input is a string value.
Output field	target_sys_rte_eb_field Output is the upper case string value without any whitespaces at the beginning and end.

Example

Input	Result
" value1 "	"VALUE1"

RTE transforms template scripts included within the Integration Commons for CMDB app

The Robust Transform Engine (RTE) transforms are templated operations included within the Integration Commons for CMDB (sn_cmdb_int_util) store app.

The following script-based RTE transforms are available in the Integration Commons for CMDB app.

CI Lookup Operation

Use to get the value of a field on an existing configuration item (CI) in the CMDB by the source native key.

Details	
Table	Robust Transform Engine CI Lookup Operation [sn_cmdb_int_util_ci_lookup_operation]
Input field	source_sys_rte_eb_field Input in order is: 1. Discovery Source 2. Source Native Key 3. CI Field The operation queries the Source [sys_object_source] table for the discovery source and the associated source native key, and then returns the CI Field value of the matching record in the target table and the associated target sys ID.
Output field	target_sys_rte_eb_field Output is the value of the field name on the CI matched by the source native key lookup or an empty string if there is no match.

The Source [sys_object_source] table is queried using the discovery source and source native key ordering by the last scan. The table iterates through the results of the query and queries the target table by the target sys ID until a valid CI is found. After a valid CI is found, the operation returns the value of the CI Field on the matching CI.

Example

Discovery Source	Source Native Key	CI Field	Result
ServiceNow	ServiceNow COMPUTER- NAME Computer-01	name	Computer-01

Cleanse Company

Use to cleanse hardware manufacturer name and add the record to the Company [core_company] table to populate a reference, when the manufacturer is not linked to a model or software (cpu_manufacturer).

Details	
Table	Robust Transform Engine Entity Cleanse Company Operation [sn_cmdb_int_util_cleansse_company_operation]
Input field	source_sys_rte_eb_field Input is a company/manufacturer name.
Output field	target_sys_rte_eb_field Output is the resulting sys_id and name of the company in core_company, concatenated by triple pipe ().
Script include function	sn_cmdb_int_util.CmdbIntegrationHardwareModelUtil().cleansseCompany(input)

If a matching record does not exist, then a new record is created in core_company so the return always includes a sys_id and name (unless the input is empty or invalid). The name is cleansed and a fuzzy lookup is done via the CmdbIntegrationCompanyModelUtil script include before the MakeAndModelJS platform API is called.

Example

Input	Result
SERVICENOW	93d4ecfac0a8000b6294d71b733977fb ServiceNow

Cleanse Hardware Model

Use to create, cleanse, or lookup a hardware model to create a reference (model_id).

Details	
Table	Robust Transform Engine Entity Cleanse Hardware Model Operation [sn_cmdb_int_util_cleansse_hardware_model_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The manufacturer name 2. The model name If either value is provided by itself, then the operation only processes what is found.
Output field	target_sys_rte_eb_field Output is the resulting sys_id and name of the company in core_company, and sys_id and name of the model in cmdb_model - all concatenated by triple pipe ().
Script include function	sn_cmdb_int_util.CmdbIntegrationHardwareModelUtil().cleansseModelAndCompany(manufacturer_in, model_in)

For either the manufacturer or model, if a matching record does not exist then a new record is created so the return always includes sys_ids and names for both records (unless the input is empty or invalid).

The manufacturer name is processed like the Cleanse Company transform and then the manufacturer name and model name are sent to the MakeAndModelJS platform API.

Example

Manufacturer Name	Model Name	Result
ServiceNow Incorporated	SERVICENOW	93d4ecfac0a8000b6294d71b733977fb ServiceNow ba29cb303710200044e0bfc8bcbe5d6d ServiceNow

Cleanse Hardware Model with Model Number

Use to create, cleanse, or lookup a hardware model to create a reference (model_id).

Details	
Table	Robust Transform Engine Entity Cleanse Hardware Model With Model Number Operatio [sn_cmdb_int_util_cleansse_hardware_model_number_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The manufacturer name 2. The model name 3. The model number 4. CI Class

Details	
	Note: The <code>CI Class</code> field is an optional input field, and when included, the <code>MakeAndModelJS</code> platform API gets the model record by querying the product model class from the <code>Model Category [cmdb_model_category]</code> table. Else, the API gets the model record from the <code>Hardware Models [cmdb_hardware_product_model]</code> table only. If either value is provided by itself, then the operation only processes what is found.
Output field	<code>target_sys_rte_eb_field</code> Output is the resulting <code>sys_id</code> and name of the company in <code>core_company</code>, and <code>sys_id</code> and name of the model in <code>cmdb_model</code> - all concatenated by triple pipe (<code> </code>).
Script include function	<code>sn_cmdb_int_util.CmdbIntegrationHardwareModelUtil().cleanseModelAndCompany(manufacturer_in, model_in)</code>

For either the manufacturer or model, if a matching record does not exist then a new record is created so the return always includes `sys_ids` and names for both records (unless the input is empty or invalid).

The manufacturer name is processed like the `Cleanse Company` transform and then the manufacturer name, model name, and model number are sent to the `MakeAndModelJS` platform API.

Example

Manufacturer Name	Model Name	Model Number	Result
ServiceNow Incorporated	SERVICENOW	BC0AA8000C56	93d4ecfac0a80 00b6294d71b733 977fb ServiceNow ba29cb3037102 00044e0bfc8bc be5d6d ServiceNow

Cleanse IP Address

Use when a field provides an IP address.

Details	
Table	Robust Transform Engine Entity Cleanse IP Operation [sn_cmdb_int_util_cleanse_ip_operation]
Input fields	source_sys_rte_eb_field Input is the IP address to cleanse.
Output field	target_sys_rte_eb_field Output is the resulting IP address, which can be empty.
Script include function	sn_cmdb_int_util.CmdbIntegrationNetworkUtil().cleansel pAddress(input)

The IP address is tested for both IPv4 and IPv6 structures along with some known derivations (an IPv4 with spaces instead of periods). If a result is found, then it is formatted and returned.

Examples

Ip	Ip Results
192.160.89.1	192.160.89.1
192.160.89.1,54.21.12.311	192.160.89.1
192 160 89 1 54 21 12 311	192.160.89.1
192-160-89-1	192.160.89.1
2001:0db8:0000:0000:0000:ff00:0042:8329	2001:0db8:0000:0000:0000:ff00:0042:8329
junk	
175912537	10.124.54.89
-1	
0	

Cleanse IP Version

Use when the source of data does not provide an IP version or when the IP version might be unreliable.

Details	
Table	Robust Transform Engine Entity Cleanse IP Version Operation [sn_cmdb_int_util_cleanse_ip_version_operation]
Input fields	source_sys_rte_eb_field Input is the IP address to cleanse.
Output field	target_sys_rte_eb_field

Details	
	Output is the resulting cmdb_ci_ip_address.ip_version lookup key (either 4, 6, or empty).
Script include function	sn_cmdb_int_util.CmdbIntegrationNetworkUtil().deriveIpVersion(input)

The input IP address value is checked for either proper IPv4 or IPv6 structure, otherwise the return is empty. This function provides no IP cleansing.

Example

Input	Result
192.160.89.1	4

Cleanse MAC Address

Use when a field provides a MAC address.

Details	
Table	Robust Transform Engine Entity Cleanse MAC Operation [sn_cmdb_int_util_cleansse_mac_operation]
Input fields	source_sys_rte_eb_field Input is the MAC address to cleanse.
Output field	target_sys_rte_eb_field Output is the resulting MAC address which can be empty.

Details	
Script include function	sn_cmdb_int_util.CmdbIntegrationNetworkUtil().cleanseMacAddress(input)

The MAC address is tested for proper structure along with some known derivations (for example, a MAC address with spaces instead of colons). If a result is found, then it is formatted and returned.

Example

Input	Result
00 0A 95 9D 68 16	00:0a:95:9d:68:16

Cleanse Operating System

Use to extract, cleanse, and format an operating system name, when the source provides an operating system value.

Details	
Table	Robust Transform Engine Entity Cleanse OS Operation [sn_cmdb_int_util_cleanse_os_operation]
Input fields	source_sys_rte_eb_field Input is the operating system name to cleanse.
Output field	target_sys_rte_eb_field Output is the resulting operating system name. The resulting operating system name is also written to the cmdb_ci_computer.os list field.

Details	
Script include function	sn_cmdb_int_util.CmdbIntegrationOsUtil().cleanseAndInsertOs(input)

Most of the current cleansing is centered on Microsoft operating system values aside from common cleansing such as fixing casing.

Examples

Operating System	Operating System Results
Windows Server 2003 R2 64 bit Edition Service Pack 2	Windows Server 2003 R2
Windows 2003	Windows 2003
Windows Vista 64 bit Edition	Windows Vista
Windows 2000 Professional Service Pack 4	Windows 2000 Professional
Windows XP Service Pack 2-3	Windows XP
Microsoft Windows Server 2003 R2 64 bit Edition Service Pack 2	Windows Server 2003 R2
Microsoft Windows 2003	Windows 2003
Microsoft Windows Vista 64 bit Edition	Windows Vista
Microsoft Windows 2000 Professional Service Pack 4	Windows 2000 Professional
Microsoft Windows XP Service Pack 2-3	Windows XP
linux ubuntu	Linux Ubuntu

Operating System	Operating System Results
Linux Ubuntu Server	Linux Ubuntu Server

Cleanse Serial Number

Use to cleanse and remove invalid serial numbers.

Details	
Table	Robust Transform Engine Entity Cleanse Serial Number Operation [sn_cmdb_int_util_cleanser_serial_number_operation]
Input fields	source_sys_rte_eb_fields Input is the serial number to cleanse.
Output field	target_sys_rte_eb_field Output is the resulting serial number.

Examples

Serial Number	Serial Number Results
ec2aa2da-5312-aa3e-804c-c35feabeda5f	ec2aa2da-5312-aa3e-804c-c35feabeda5f
1045-1209-6738-4668-7696-2783	1045-1209-6738-4668-7696-2783

Cleanse Serial Number With Alternate Value Return if Invalid

Use to cleanse Serial Number and verify the length of the cleansed serial number. Returns the alternate value if the length of serial number is invalid.

Details	
Table	Robust Transform Engine Entity Cleanse Serial Number With Alternate Value Return [sn_cmdb_int_util_cle\$anse_serial_number_with_alternate_value_return_if_invalid]
Input fields	source_sys_rte_eb_fields Inputs: serial number, Alternate value
Output field	target_sys_rte_eb_field Output: Cleansed serial number if the length is valid, Alternate value otherwise.
Script include function	global.SerialNumberManager().isValid(input)

The operation cleanses the serial number and checks if the cleansed serial number length is greater than or equal to the value specified in the sn_cmdb_int_util.minimum_valid_serial_number_length system property. The default value of the property is set to 7.

Cleanse IMEI Number With Alternate Value Return if Invalid

Use to cleanse the IMEI number and verify the length of the cleansed IMEI number. Returns the alternate value if the length of IMEI number is invalid.

Details	
Table	Robust Transform Engine Entity Cleanse IMEI With Alternate Value Return if Inval

Details	
	[sn_cmdb_int_util_cleans_imei_with_alternate_value_return_if_invalid]
Input fields	source_sys_rte_eb_fields Inputs: IMEI number, Alternate value
Output field	target_sys_rte_eb_field Output: IMEI number if the length is valid, Alternate value otherwise.
Script include function	None

The operation cleanses the IMEI number and checks if the cleansed IMEI number length is equal to the value specified in the sn_cmdb_int_util.valid_imei_number_length system property. The default value of the property is set to 15.

Cleanse Software Model

Use to cleanse and create a software model. Also, to create manufacturer and software model if they do not exist and follow with a split operation.

Details	
Table	Robust Transform Engine Entity Cleanse Software Model Operation [sn_cmdb_int_util_cleans_software_model_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The manufacturer name

Details	
	2. The software name 3. The software version (not required) If only manufacturer or name is provided, then only those values are processed and returned.
Output field	target_sys_rte_eb_field Output is the resulting sys_id and name of the company in core_company, the cleansed software name, and the cleansed software version all concatenated by a triple pipe ().
Script include function	sn_cmdb_int_util.CmdbIntegrationSoftwareModelUtil().cleanseSoftwareModel(company, model, version)

If a matching manufacturer record does not exist, then a new record is created so the return always includes the sys_id and name for the manufacturer (if the manufacturer is not empty or invalid).

The manufacturer name is processed the same as in the Cleanse Company transform and then the manufacturer name is sent to the MakeAndModelJS platform API.

The software name and version are cleansed and formatted and returned. The version is removed from the software name if present.

Examples

Manufacturer	Software Name	Software Version	Results
Dell Inc.			
	NoManufacturer		

Manufacturer	Software Name	Software Version	Results
		1.0.0.0	
Dell Inc.	DataEngine	1.0.17.2	b7e7d7d8c0a8016900a5d7f291acce5c Dell Inc. DataEngine 1.0.17.2
GenuineIntel	TestSoftware	1.0.0.1	7aad6d00c611228400f00e0f80b67d2d Intel TestSoftware 1.0.0.1
Dell Inc.	TestSoftware	232	b7e7d7d8c0a8016900a5d7f291acce5c Dell Inc. TestSoftware 232
Dell Inc.	TestSoftware	123.0.0.0	b7e7d7d8c0a8016900a5d7f291acce5c Dell Inc. TestSoftware 123.0
America Online	TestSoftware	1.0.0.0	0c43d035c61122750000251553f6f8e8 America Online TestSoftware 1.0
America Online	TestSoftware	1.0.0.0	0c43d035c61122750000251553f6f8e8 America Online TestSoftware 1.0
dell	LowerCase	1.0.0.0	b7e7d7d8c0a8016900a5d7f291acce5c Dell Inc. LowerCase 1.0
Dell Corporation. Incorporated, Corp.	TestSoftware	1.0.0.0	b7e7d7d8c0a8016900a5d7f291acce5c Dell Inc. TestSoftware 1.0

Manufacturer	Software Name	Software Version	Results
Microsoft	Microsoft SQL Server 2016 Enterprise	2.0.0	0e8b8e650a0a0b3b004f285ffbb1a4fc Microsoft Microsoft SQL Server 2016 Enterprise 2.0
Dell Computer	DataEngine	1.0.17.2	b7e7d7d8c0a8016900a5d7f291acce5c Dell Inc. DataEngine 1.0.17.2
Adobe	TestSoftware	1	b7e8b5c4c0a80169008b49e468920048 Adobe Systems TestSoftware 1.0

Create Software Instance Name

Use when a hardware name, software name, and software version is provided, to create a new software instance name.

Details	
Table	Robust Transform Engine Entity Create Software Instance Name Operation [sn_cmdb_int_util_create_software_instance_name_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The hardware name 2. The software name 3. The software version (not required)

Details	
Output field	target_sys_rte_eb_field Output is the software instance name.
Script include function	sn_cmdb_int_util.CmdbIntegrationSoftwareModelUtil().createSoftwareInstanceName(hw_name_in, sw_name_in, sw_version_in)

Examples

Hardware Name	Software Name	Software Version	Results
computer1	microsoft	2.0.1	microsoft 2.0.1-computer1
computer2	adobe		adobe-computer2
computer3	adobe	2.1	adobe 2.1-computer3
hw2	sw3	301	sw3 301-hw2

Derive CI Class from Model

Use when processing a computer record and a model is provided but the class of the computer is ambiguous otherwise. Can be used along with other Derive CI Class transforms.

Details	
Table	Robust Transform Engine Entity Derive Class From Model Operation [sn_cmdb_int_util_derive_class_from_model_operation]
Input fields	source_sys_rte_eb_fields Input in order is:

Details	
	1. The model name 2. The current class name
Output field	target_sys_rte_eb_field Output is the resulting class name.
Script include function	sn_cmdb_int_util.CmdbContextIntegrationClassUtil().deriveClassNameFromModelInput(model_in, class_in)

Does not return a value of a class which is higher in the class hierarchy (a parent class) than the provided input class. For example, does not return cmdb_ci_computer if the input is cmdb_ci_server. Looks only at the cmdb_ci_computer hierarchy, going through cmdb_ci_server (cmdb_ci_computer, cmdb_ci_server, children of cmdb_ci_server).

Currently looks for Server, Windows Server, and Linux Server indicators in the model.

Examples

Model	Class	Class Results
window server	cmdb_ci_computer	cmdb_ci_win_server
Microsoft server	cmdb_ci_computer	cmdb_ci_win_server
linux server	cmdb_ci_computer	cmdb_ci_linux_server
Microsoft server	cmdb_ci_server	cmdb_ci_win_server
linux server	cmdb_ci_server	cmdb_ci_linux_server
Red hat server	cmdb_ci_server	cmdb_ci_linux_server
Arch server	cmdb_ci_server	cmdb_ci_linux_server

Model	Class	Class Results
Centos server	cmdb_ci_server	cmdb_ci_linux_server
Debian server	cmdb_ci_server	cmdb_ci_linux_server
Fedora server	cmdb_ci_server	cmdb_ci_linux_server
Suse server	cmdb_ci_server	cmdb_ci_linux_server
Oracle server	cmdb_ci_server	cmdb_ci_linux_server
Rhel server	cmdb_ci_server	cmdb_ci_linux_server
Ubuntu server	cmdb_ci_server	cmdb_ci_linux_server
Junk		
	cmdb_ci_server	cmdb_ci_server
Junk server	cmdb_ci_computer	cmdb_ci_server
Junk	cmdb_ci_computer	cmdb_ci_computer

Derive CI Class from Native Class Identifier

Use when processing a computer record and a native class indicator is provided but the class of the computer is ambiguous otherwise. Can be used along with other Derive CI Class transforms.

Details	
Table	Robust Transform Engine Entity Derive Class From Native Value Operation [sn_cmdb_int_util_derive_class_from_native_value_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The native class identifier

Details	
	2. The current class name
Output field	target_sys_rte_eb_field Output is the resulting class name.
Script include function	sn_cmdb_int_util.CmdbContextIntegrationClassUtil().deriveClassNameFromNativeValue(native_id_in, class_in)

Does not return a value of a class which is higher in the class hierarchy (a parent class) than the provided input class. For example, does not return cmdb_ci_computer if the input is cmdb_ci_server. Looks only at the cmdb_ci_computer hierarchy, going through cmdb_ci_server (cmdb_ci_computer, cmdb_ci_server, children of cmdb_ci_server).

Currently looks for Server, Windows Server, and Linux Server indicators in the native identifier.

Examples

Native Class	Class	Class Results
window server	cmdb_ci_computer	cmdb_ci_win_server
Microsoft server	cmdb_ci_computer	cmdb_ci_win_server
linux server	cmdb_ci_computer	cmdb_ci_linux_server
Microsoft server	cmdb_ci_server	cmdb_ci_win_server
linux server	cmdb_ci_server	cmdb_ci_linux_server
Red hat server	cmdb_ci_server	cmdb_ci_linux_server
Arch server	cmdb_ci_server	cmdb_ci_linux_server
Centos server	cmdb_ci_server	cmdb_ci_linux_server

Native Class	Class	Class Results
Debian server	cmdb_ci_server	cmdb_ci_linux_server
Fedora server	cmdb_ci_server	cmdb_ci_linux_server
Suse server	cmdb_ci_server	cmdb_ci_linux_server
Oracle server	cmdb_ci_server	cmdb_ci_linux_server
Rhel server	cmdb_ci_server	cmdb_ci_linux_server
Ubuntu server	cmdb_ci_server	cmdb_ci_linux_server
Junk		
	cmdb_ci_server	cmdb_ci_server
Junk server	cmdb_ci_computer	cmdb_ci_server
Junk	cmdb_ci_computer	cmdb_ci_computer

Derive CI Class from Operating System

Use when processing a computer record and an operating system is provided but the class of the computer is ambiguous otherwise. Can be used along with other Derive CI Class transforms.

Details	
Table	Robust Transform Engine Entity Derive Class From OS Operation [sn_cmdb_int_util_derive_class_from_os_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The operating system name 2. The current class name

Details	
Output field	target_sys_rte_eb_field Output is the resulting class name.
Script include function	sn_cmdb_int_util.CmdbIntegrationClassUtil().deriveClassNameFromOsName(os_in, class_in)

Does not return a value of a class which is higher in the class hierarchy (a parent class) than the provided input class. For example, does not return cmdb_ci_computer if the input is cmdb_ci_server. Looks only at the cmdb_ci_computer hierarchy, going through cmdb_ci_server (cmdb_ci_computer, cmdb_ci_server, children of cmdb_ci_server).

Currently looks for Server, Windows Server, and Linux Server indicators in the operating system name.

Examples

Operating System	Class	Class Results
window server	cmdb_ci_computer	cmdb_ci_win_server
Microsoft server	cmdb_ci_computer	cmdb_ci_win_server
linux server	cmdb_ci_computer	cmdb_ci_linux_server
Microsoft server	cmdb_ci_server	cmdb_ci_win_server
linux server	cmdb_ci_server	cmdb_ci_linux_server
Red hat server	cmdb_ci_server	cmdb_ci_linux_server
Arch server	cmdb_ci_server	cmdb_ci_linux_server
Centos server	cmdb_ci_server	cmdb_ci_linux_server
Debian server	cmdb_ci_server	cmdb_ci_linux_server

Operating System	Class	Class Results
Fedora server	cmdb_ci_server	cmdb_ci_linux_server
Suse server	cmdb_ci_server	cmdb_ci_linux_server
Oracle server	cmdb_ci_server	cmdb_ci_linux_server
Rhel server	cmdb_ci_server	cmdb_ci_linux_server
Ubuntu server	cmdb_ci_server	cmdb_ci_linux_server
Junk		
	cmdb_ci_server	cmdb_ci_server
Junk server	cmdb_ci_computer	cmdb_ci_server
Junk	cmdb_ci_computer	cmdb_ci_computer

Derive Virtual From Hardware Model

Use when processing a computer record that may be virtual, a hardware model is provided, and the virtual status is ambiguous. Can be used along with other Derive Virtual From transforms.

Details	
Table	Robust Transform Engine Entity Derive Virtual From Model Operation [sn_cmdb_int_util_derive_virtual_from_model_operation]
Input fields	source_sys_rte_eb_fields Input in order is: 1. The hardware model name 2. The current virtual flag value
Output field	target_sys_rte_eb_field

Details	
	Output is the resulting virtual flag (true/false). If the current virtual flag is 'true', the result is true. Otherwise the result is 'true' or 'false'.
Script include function	sn_cmdb_int_util.CmdbIntegrationVirtualDetectionUtil().detectVirtualFromModelName(model_in, is_virtual_in)

Looks for indicators in the model name for a virtual device (VMware).

Examples

Hardware Model	Virtual Flag	Virtual Flag Results
thinkpad	true	true
thinkpad	false	false
thinkpad		false
vmware inc	true	true
	true	true
	false	false
		false

Derive Virtual From Native Indicator

Use when processing a computer record that may be virtual, a virtual indicator is provided by the source, and the virtual status is ambiguous. Can be used along with other Derive Virtual From transforms.

Details	
Table	sn_cmdb_int_util_derive_virtual_from_native_value_operation
Input fields	source_sys_rte_eb_fields Input in order is: 1. The native indicator 2. The current virtual flag value
Output field	target_sys_rte_eb_field Output is the resulting virtual flag (true/false). If the current virtual flag is 'true', the result is true. Otherwise the result is 'true' or 'false'.
Script include function	sn_cmdb_int_util.CmdbIntegrationVirtualDetectionUtil().detectVirtualFromNativeIdentifier(native_in, is_virtual_in)

Tests native indicator against a list of common values and looks for a 'true' boolean indicator.

Examples

Native Virtual Value	Virtual Flag	Virtual Flag Results
virtual	false	true
virtual		true
virtual	true	true
y	false	true
y		true

Native Virtual Value	Virtual Flag	Virtual Flag Results
y	true	true
yes	false	true
yes		true
yes	true	true
true	false	true
true		true
true	true	true
t	false	true
t		true
t	true	true
other	false	false
other		false
other	true	true
not virtual	false	false
not virtual		false
not virtual	true	true
	false	false
		false
	true	true

Derive Virtual From Serial Number

Use when processing a computer record that may be virtual, a serial number is provided by the source, and the virtual status is ambiguous. Can be used along with other Derive Virtual From transforms.

Details	
Table	sn_cmdb_int_util_derive_virtual_from_serial_number_operation
Input fields	source_sys_rte_eb_fields Input in order is: 1. The serial number 2. The current virtual flag value
Output field	target_sys_rte_eb_field Output is the resulting virtual flag (true/false). If the current virtual flag is 'true', the result is true. Otherwise the result is 'true' or 'false'.
Script include function	sn_cmdb_int_util.CmdbIntegrationVirtualDetectionUtil().detectVirtualFromSerialNumber (serial_in, is_virtual_in)

Looks for indicators in the serial number for a virtual device (VMware).

Examples

Serial Number	Virtual Flag	Virtual Flag Results
123	true	true
123	false	false
123		false

Serial Number	Virtual Flag	Virtual Flag Results
vmware-123	true	true
	true	true
	false	false
		false

Extract and Scale by Units

Use when the source has numerical values that need to be scaled and numerical value with an input such as 2048Mb. The source does not always provide the units so it may be required to calculate or guess the units being provided. The target units depend on the target field in the CMDB. If not specified, the decimal place field is set at 2 by default.

Note: This field is case sensitive.

Details	
Table	sn_cmdb_int_util_extract_and_scale_by_units_operation
Input fields	source_sys_rte_eb_fields
Output field	target_sys_rte_eb_field
Script include function	sn_cmdb_int_util.CmdblIntegrationExtractScaleUnitUtil().extractAndScaleUnits(input,defaultUnit,outputUnit,decimalPlaces)

Examples

Input Value	Default Unit	Output Unit	Result
2048Mb	Mb	GB	2GB
17179869184	B	GB	16GB

First Non Null Value

Use when you have a list of fields providing similar information that must map to a single field and you want to rank the order in which they can provide those values.

For Example Internally in SolarWinds, there is a hierarchy of tables that are join. In one example, a computer's name could come from the child most table or any of that tables parents but each of those is a separate field in the pull. Starting with the most specific table, the values are searched for the first instance of a name value.

Details	
Table	sn_cmdb_int_util_first_non_null_operation
Input fields	source_sys_rte_eb_fields Input is a list of fields of any length.
Output field	target_sys_rte_eb_field Output is the value from the first field in the list that doesn't have a null (or empty) value.
Script include function	sn_cmdb_int_util.CmdbIntegrationFirstNonNullValueUtil().firstNonNullValue(batch[i])

Example

Field 1	Field 2	Field 3	Result
	foo	foo2	foo

Process Name, Domain, FQDN, DNS set

Use when the source provides name, domain, FQDN, or DNS information. Can be used for only a subset of these (if for example, the source only provides name and domain). In the case that a source

only provides fields that are lower in the input list (FQDN) the CmdbIntegrationHardwareNameUtil script include can be called from a script operation to minimize having to create empty dummy fields.

Details	
Table	sn_cmdb_int_util_process_name_set_operation
Input fields	source_sys_rte_eb_fields Takes up to four Input fieldss (any additional fields are ignored), in the following order: 1. Name 2. Domain 3. FQDN 4. DNS You don't have to provide all four input values, but you must provide those values in the specified order. If for example, you only want to cleanse domain, you must provide a name attribute, even it if empty.
Output field	target_sys_rte_eb_field Output is a concatenated set of values in the same order, using a triple pipe (): {name} {domain} {fqdn} {dns}
Script include function	sn_cmdb_int_util.CmdbIntegrationHardwareNameUtil().processNameDomainFqdnDnsSet(name, domain, fqdn, dns)

FQDN and DNS are first processed to see if their formats are correct. FQDN has an additional discovery regex it must pass (via properties):
glide.discovery.fqdn.regex - default : ^([^.]+)\.\((?:[^.]+\|\.)+\)\$

Possible name and domain values are extracted if possible. When name and domain are processed, if there is no FQDN, a value is generated if possible. A resulting name value is also modified using the following discovery flags:

1. glide.discovery.hostname.case – default: No change. Can be set to 'Lower case', 'Upper case', 'No change'
2. glide.discovery.hostname.include_domain – default: false. If 'true' the domain is added to the final name value

Examples

Name	Domain	FQDN	DNS	Results
myName	other.net	otherName. other.net	mycomp.ser vicenow.co m	myName other.net otherName. other.net mycomp.ser vicenow.co m
na	other.net	otherName. other.net	mycomp.ser vicenow.co m	otherName other.net otherName. other.net mycomp.ser vicenow.co m
			servicenow. com	 servicenow. com
			name.servic enow.com	name servicenow.

Name	Domain	FQDN	DNS	Results
				com name.servic enow.com name.servic enow.com
		name.servic enow.com		name servicnow. com name.servic enow.com
		name.servic enow.com		name servicnow. com name.servic enow.com
name	servicnow. com			name servicnow. com name.servic enow.com

Process FQDN

Use when the source provides a suspected FQDN value but no other naming fields such as name, domain, or DNS.

Details	
Table	sn_cmdb_int_util_process_fqdn_operation
Input fields	source_sys_rte_eb_field

Details	
	Input is a single field containing an FQDN.
Output field	target_sys_rte_eb_field Output is a concatenated set of values in the same order using a triple pipe (): {name} {domain} {fqdn} {dns}
Script include function	sn_cmdb_int_util.CmdbIntegrationHardwareNameUtil().processNameDomainFqdnDnsSet(", ", fqdn, ")

The processing follows the same logic as the 'Process Name, Domain, FQDN, DNS set' transform except that only FQDN is used as an input.

Example

Input	Result
mycomputer.servicenow.com	mycomputer servicenow.com mycomputer.servicenow.com

Scale Unit

Use when the source has numerical inputs that must be scaled. The source does not always provide the current units so it may be required to calculate or guess the units being provided. The target units depend on the field being targeted in the CMDB.

Details	
Table	sn_cmdb_int_util_scale_unit_operation
Input fields	source_sys_rte_eb_fields Inputs in order are:

Details	
	1. The input value 2. The current units 3. The target units
Output field	target_sys_rte_eb_field Output is the input value scaled from the current units to the target units. If no units are found for the current units, then the input value is returned. If no current or target units are found the input is returned as the output.
Script include function	sn_cmdb_int_util.CmdbIntegrationScaleUnitUtil().scaleUnits(input_value, input_unit, output_unit)

Example

Input	Result
• Input Field 1: 1 • Input Field 2: GB • Input Field 3: MB	1024

Software Bundle ID Lookup

Use when a source, such as Jamf, does not provide the software publisher but does provide a Mac software bundle ID. Software Bundle ID Lookup looks up records in the Bundleid Lookup [sn_cmdb_int_util_bundleid_lookup] table by bundle_id. If a record with the specified bundle_id exists, it extracts the respective software publisher. Otherwise, it creates a new record which will be queried the next time the Lookup Mac Software Bundle IDs data source runs.

Details	
Table	sn_cmdb_int_util_software_bundle_id_lookup_operation
Input fields	- source_sys_rte_eb_field - Bundle ID
Output field	target_sys_rte_eb_field Output is the resulting artist name, track name, and seller name, all concatenated by a triple pipe (), or an empty string if no match is found.
Script include function	sn_cmdb_int_util.CmdbIntegrationSoftwareBundleIdLookup.lookupSoftware(bundleId)

Example

Input	Result
Input Field 1: com.microsoft.Word	Microsoft Corporation Microsoft Word Microsoft Corporation

User Lookup

Use to look up a user in the User [sys_user] table by user name or email, attempting to match in the following order:

1. The User Name matching the user_name attribute.
2. The Email matching the email attribute.
3. If nothing is matching, it returns empty.

Details	
Table	sn_cmdb_int_util_user_lookup_operation
Input fields	source_sys_rte_eb_fields Inputs in order are: 1. User Name 2. Email (Optional)
Output field	target_sys_rte_eb_field sysId of the sys_user.
Script include function	sn_cmdb_int_util.CmdbIntegrationUserLookup.lookupUser(username, email)

Examples

Input	Result
Input Field 1: abel.tuter	62826bf03710200044e0bfc8bcbe5df1
• Input Field 1: atuter • Input Field 2: abel.tuter@example.com	62826bf03710200044e0bfc8bcbe5df1

Configuring the ADM adapter for Service Graph Connectors

You can configure the Application Dependency Mapping (ADM) adapter to populate running processes, TCP connections, and applications into CMDB.

As a user with the admin role, you can use the ADMHelper script include to configure the ADM adapter that populates running processes, TCP connections, and applications into CMDB. The ADMHelper script include is available within the Integration Commons for CMDB (sn_cmdb_int_util) store app. The ADMHelper script include invokes the ApplicationDependencyMapping script include that is available within the Discovery and Service Mapping Patterns application (sn_itom_pattern).

The ADM adapter requires the inputs as discussed in the following table:

Inputs for the ADM adapter

Input	Input type
Running Process Details	Required
Computer Sys Id	Required
TCP connection details	Optional
ADM Properties	Optional

For interpreting and populating the CIs, the ADM processor requires the input data in a specific format. Ensure that the keys are formed as shown in the following example:

Example keys for ADM processor

TCP connections data	Running process
<pre>[{ "pid": "1068", "local_ip": "127.0.0.1", "local_port": "199", "ip": "0.0.0.0", "port": "199", "state": "LISTEN", "type": "on" }</pre>	<pre>[{ "pid": "1", "ppid": "0", "command": "/usr/lib/systemd/systemd", "name": "systemd", "parameters": "--switched-root --system --deserialize 21" }</pre>

TCP connections data	Running process
<pre>{ }</pre>	<pre>]</pre>

The ApplicationDependencyMapping script include processes TCP connections and running process data and populates the following tables:

- TCP Connections [cmdb_tcp]
- Running Process [cmdb_running_process]
- Application [cmdb_ci_appl]

Note: After the data is populated into the TCP Connections [cmdb_tcp] and Running Process [cmdb_running_process] tables, the ApplicationDependencyMapping script include reconciles the TCP connections and running process data for populating the Application [cmdb_ci_appl] table based on the data in the Network Adapter [cmdb_ci_network_adapter] and IP Address [cmdb_ci_ip_address] tables. Classified processes are added to the corresponding child class in the Application [cmdb_ci_appl] table. The addition of unclassified processes to the Application [cmdb_ci_appl] table depends on the system property value.

Accessing the connection details of Service Graph Connectors

The common connection framework (CCF) included within the Integration Commons for CMDB (sn_cmdb_int_util) store app enables accessing connection details of Service Graph Connectors in a single view.

With the CCF, you can access all the connections used by Service Graph Connectors or the connections that are specific to a Service Graph Connector. The connection details include the connection alias, connection properties, data sources, and scheduled data imports associated with a connection for a Service Graph Connector.

Viewing connections

To view the connection details for all Service Graph Connectors that use the CCF, navigate to **All > Service Graph Connectors > Connections** from the application navigator.

If a Service Graph Connector uses CCF, you can also view the connections specific to the connector. To view the connection details, open the Service Graph Connector from the application navigator, access its Connections module to view the Service Graph Connections list, and then select the connection name.

The Service Graph Connections page displaying the connection details also includes the **Test Connection** related link that you can select to test the connection. If the connection is successful, a success message is displayed. Else, an error message with the reason of failure is displayed.

CCF tables

The CCF uses the following tables:

Service Graph Connections [sn_cmdb_int_util_service_graph_connection]

Stores the connection records for all the Service Graph Connectors that use the CCF. The Service Graph Connections [sn_cmdb_int_util_service_graph_connection] table also includes other CCF tables as related lists.

Properties [sn_cmdb_int_util_service_graph_connection_property]

Stores the connection properties associated with a specific connection.

Service Graph Connection Data Sources [sn_cmdb_int_util_service_graph_connection_data_source]

Stores the many-to-many (m2m) connection details between the Service Graph Connections [sn_cmdb_int_util_service_graph_connection] table and the data sources for a Service Graph Connector.

Service Graph Connection Scheduled Data Imports [sn_cmdb_int_util_service_graph_connection_scheduled_data_import]

Stores the m2m connection details between the Service Graph Connections [sn_cmdb_int_util_service_graph_connection] table and the scheduled data imports for a Service Graph Connector.

Service Graph Connection Test Script **[sn_cmdb_int_util_service_graph_connection_test_script]**

Stores records that include scripts for triggering the test connection process for Service Graph Connectors.

Managing CMDB data deletion

The Integration Commons for CMDB (sn_cmdb_int_util) store app uses post-script and transform map methods to remove imported Configuration Management Database (CMDB) records that were deleted later from the source third-party application.

Post-script method

The post-script method checks the Source [sys_object_source] table for configuration items (CIs) not updated in the last run for a specific source feed. It then removes any non-updated CIs from the associated target CMDB tables synchronously, handling the removal of target records one at a time.

Most Service Graph Connectors use the post-script method to manage the removal of any CMDB records.

The post-script method uses the RecordRemoval script to handle various record deletion strategies. Hard deletion involves permanently and completely removing records from the CMDB, making them unrecoverable and suitable for data that is no longer needed. Soft deletion marks records as deleted without actually removing them from the CMDB, allowing for potential recovery and preserving historical data. Optimized soft deletion, managed by the Identification and Reconciliation Engine (IRE), enhances this soft deletion approach by using a specialized table, such as the extension attributes table, and applying reconciliation rules to manage deleted records more efficiently. By default, the table can hold up to 100 records at a time, but as a user with the admin role, you can adjust this limit by setting the sn_cmdb_int_util.ire_payload_items_size system property. This property is not available by default and you must create this property of type integer within the Integration Commons for CMDB (sn_cmdb_int_util) store app. For more information, see [Add a system property](#).

Transform map-based method

The transform map-based method checks the Source [sys_object_source] table for configuration items (CIs) not updated in the last run for a specific source feed. It then utilizes the multi-threaded deletion approach to remove any non-updated CIs from the associated target CMDB tables concurrently, handling the removal of all target records within an import set at once. Multiple import sets are created to store target record data within the Integration Commons Remove Record [sn_cmdb_int_util_remove_record] staging table enabling multi-threaded deletion of records. Therefore, reducing the data processing time.

The Integration Commons Remove Record [sn_cmdb_int_util_remove_record] staging table tracks non-updated CIs by storing target record details including the discovery source, source feed, target table sys_id, and target table name of these CIs. The on-start script defined within the Remove Record Transform Map then deletes the records from the associated target tables.

The transform map-based method is used by a Service Graph Connector to improve the data processing time. For example, the SG-Tanium Remove Software data source within the Service Graph Connector for Tanium removes software data using the transform map-based method. See [Service Graph Connector for Tanium](#).

Partition size computation for parallel loading in Integration Commons for CMDB

Integration Commons for CMDB supports computed partition size for the parallel loading framework, if supported in a Service Graph Connector.

The parallel loading feature of ServiceNow AI Platform, executes data load using multiple threads. The partition size for a data source is computed by the Integration Commons for CMDB application used by the parallel loading feature. This partition size confirms that the workload is evenly distributed among the available threads. By balancing the data load, a data source within a Service Graph Connector can manage data loading uniformly across the available nodes, reducing processing time and improving overall efficiency. This approach promotes efficient data loading, minimizes latency, and optimizes the performance of data sources.

The partition size computation for the parallel loading feature within the Integration Commons for CMDB application is based on the following calculations:

- Partition size calculation
- Parallel jobs calculation

Partition size calculation

The partition size for parallel loading in a data source is determined by both the number of active primary nodes on your ServiceNow instance and the total volume of data to be loaded by the data source using the following formula:

$$\text{Partition size} = \text{Total data volume} / (10\% * \text{Active primary nodes})$$

For example, if there are 4 active primary nodes and the data volume to be loaded is 1000 units, the partition size is calculated as:

$$\text{Partition size} = 1000 / (10\% * 4) = 1000 / 0.4 = 25$$

Parallel jobs calculation

Parallel jobs enable distributing the workload evenly across multiple jobs, reducing the data loading time and improving the efficiency of data retrieval. The calculation of the number of parallel jobs created for a partition is based on the following formula:

$$\text{Number of parallel jobs} = \text{Total data volume} / \text{Partition size}$$

In the earlier example, where the total data volume is 1000 units and the partition size is 25, the number of parallel jobs created is calculated as:

$$\text{Number of parallel jobs} = 1000 / 25 = 40$$

Quick start tests for Integration Commons for CMDB

Validate that integrations for CMDB pass validation and still work after you make any configuration changes such as applying an upgrade or developing an application.

Danger: By default, the system property that is used to run automated tests is disabled to prevent you from accidentally running these tests on a production system. To avoid data corruption or an outage, run tests only on development, test, and other non-production instances. See [Enable or disable executing Automated Test Framework tests](#).

Integration Commons for CMDB

CMDB INT: CMDB Integrations Validation test suite

Test suite to verify the integrity of an integration using multiple tests.

Test	Description	Release version
CMDB INT: Set Test Session Application	Modify the run server-side script to set an application name so that you can test only one integration. Otherwise, all integrations installed will be tested.	Paris
CMDB INT: Test Against Source Analysis	Test an integration against the values in the CMDB Integration Source Analysis [sn_cmdb_int_util_cmdb_integration_source_analysis] table.	Paris
CMDB INT: Validate Application Feed	Validate all application feeds in an integration.	Paris
CMDB INT: Validate Discovery Source	Validate that the discovery source exists.	Paris

Test	Description	Release version
CMDB INT: Validate Entity Mappings	Validate all entity mappings of an integration.	Paris
CMDB INT: Validate Fields	Validate fields for CMDB Integrations.	Paris
CMDB INT: Validate Lookups	Validate CMDB integration lookups.	Paris
CMDB INT: Validate Mandatory Operations	Validate that all integrations for mandatory operations exist for mapped fields.	Paris
CMDB INT: Validate Operations	Validate all operations for an integration.	Paris
CMDB INT: Validate References	Validate CMDB integration references.	Paris
CMDB INT: Validate Related Entries	Validate all related classes against the data dictionary for related entries.	Paris
CMDB INT: Validate Relationships	Validate CMDB integration relationships.	Paris

To learn more about Integration Commons for CMDB, see [Integration Commons for CMDB](#).

Related topics

- [Quick start tests](#)